ESP32 Co-Processor

STM32 OTA Firmware Updater

PlatformIO + ESP-IDF | XModem Protocol

Overview

This project enables an ESP32 to act as a co-processor that remotely updates the firmware of an STM32 microcontroller via XModem protocol over UART. The ESP32 opens a Wi-Fi Access Point and serves a web interface for uploading the STM32 .bin file, then handles the full bootloader entry sequence and XModem transfer automatically.

System Architecture

Component	Role
ESP32	Wi-Fi AP + HTTP server + XModem transmitter
STM32 (target)	Custom XModem bootloader receiver
PC / Phone	Connects to ESP32 AP, uploads .bin via browser
SPIFFS	Temporary storage for firmware binary on ESP32

Hardware Connections

ESP32 GPIO	STM32 Pin	Function	Notes
GPIO 32	PA0 (BOOT0)	Bootloader trigger	HIGH = enter bootloader
GPIO 33	NRST	Reset	Active LOW pulse
GPIO 17 (TX)	PA10 USART1 RX	UART data	115200 baud
GPIO 16 (RX)	PA9 USART1 TX	UART data	115200 baud
GND	GND	Common ground	Required

Warning: Do NOT use GPIO 25/26 — these are DAC1/DAC2 on ESP32 and will cause resets during boot.

GPIO 34, 35, 36, 39 are input-only and cannot be used for output signals.

Project Structure

```
stm32_ota/
├── platformio.ini          # PlatformIO config
├── partitions.csv          # Custom flash partition table
├── sdkconfig.esp32dev      # ESP-IDF SDK config
└── src/
    ├── main.c              # App main: AP + HTTP server
    ├── stm32_updater.c     # XModem transmitter + GPIO control
    └── stm32_updater.h     # Public API
```

Configuration

platformio.ini

```
[env:esp32dev]
platform    = espressif32
board       = esp32dev
framework   = espidf
board_build.partitions = partitions.csv
monitor_speed = 115200
```

partitions.csv

Name	Type	SubType	Offset	Size
nvs	data	nvs	0x9000	0x6000
phy_init	data	phy	0xF000	0x1000
factory	app	factory	0x10000	0x180000
spiffs	data	spiffs	0x190000	0x60000

SPIFFS partition is 384 KB — sufficient for STM32 firmware binaries up to 256 KB.

Wi-Fi Access Point

Parameter	Value
SSID	ESP32-STM32-OTA
Password	12345678
IP Address	192.168.4.1
Security	WPA2-PSK

Pin Definitions (stm32_updater.h)

```
#define STM32_BOOT_PIN      GPIO_NUM_32   // PA0 bootloader trigger
#define STM32_NRST_PIN      GPIO_NUM_33   // NRST reset
#define STM32_UART_PORT     UART_NUM_2
#define STM32_UART_TX       GPIO_NUM_17
#define STM32_UART_RX       GPIO_NUM_16
#define STM32_UART_BAUD     115200
```

Timing Parameters

Define	Value	Description
RESET_ASSERT_MS	20 ms	NRST low pulse width
BOOT_SETUP_MS	10 ms	PA0 setup time before reset
BOOTLOADER_READY_MS	3500 ms	Wait for STM32 bootloader NAK_INIT_DELAY (3000 ms) + margin
XMODEM_TIMEOUT_MS	3000 ms	Per-byte receive timeout (matches STM32 side)
XMODEM_MAX_RETRY	10	Max block retransmit attempts

Operation

Bootloader Entry Sequence

The ESP32 enters the STM32 custom bootloader using the following hardware sequence:

```
1. Assert PA0 (BOOT0) HIGH   → GPIO32 = 1
2. Assert NRST LOW           → GPIO33 = 0  (20 ms)
3. Release NRST HIGH         → GPIO33 = 1
4. Wait 3500 ms              → STM32 bootloader sends NAK
5. Flush UART RX buffer      → discard any garbage bytes
6. Begin XModem handshake
```

XModem Transfer

- Protocol: XModem with 8-bit checksum (classic mode)
- Block size: 128 bytes
- Padding: last block padded with 0x1A (SUB character)
- Block numbers wrap: 0xFF -> 0x00 automatically (uint8_t)
- File is streamed from SPIFFS — no large RAM buffer required

Bootloader Exit Sequence

```
1. De-assert PA0 (BOOT0)      → GPIO32 = 1 (already high, no change)
2. Assert NRST LOW            → GPIO33 = 0  (20 ms)
3. Release NRST HIGH          → GPIO33 = 1
   STM32 boots into application
```

Web Upload Flow

```
PC/Phone  ──Wi-Fi──▶  ESP32 AP (192.168.4.1)
                          |
               HTTP POST /upload (.bin file)
                          |
               Save to SPIFFS /spiffs/stm32fw.bin
                          |
               stm32_enter_bootloader()
                          |
               XModem stream SPIFFS → STM32 UART
                          |
               stm32_exit_bootloader()
                          |
               HTTP response: 'Flash success!'
```

Usage

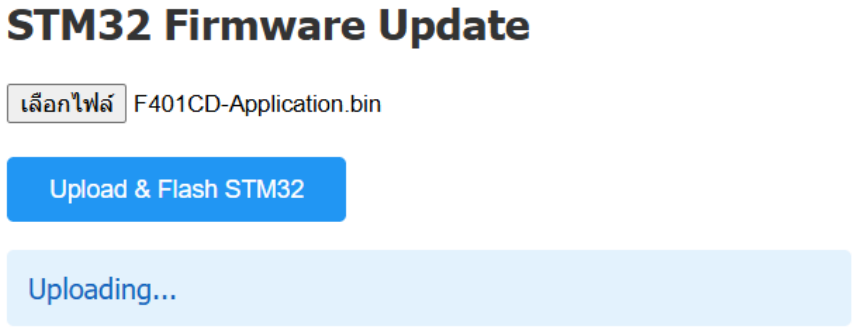
Build and Flash ESP32

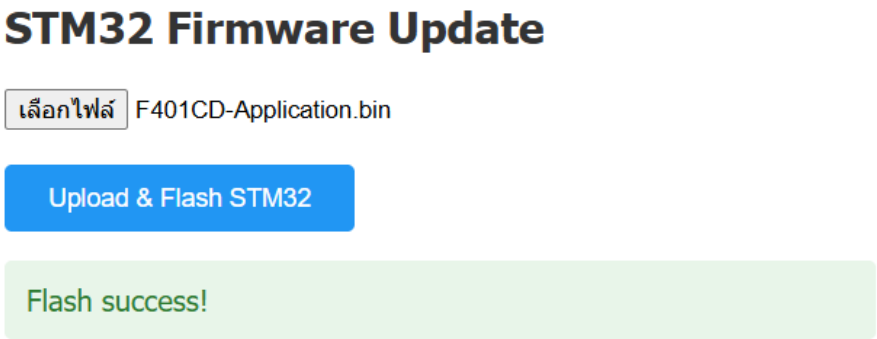
```
# Build
pio run

# Flash ESP32 firmware
pio run --target upload

# Monitor serial output
pio device monitor
```

Update STM32 Firmware

1. Power on ESP32
2. Connect PC or phone to Wi-Fi: "ESP32-STM32-OTA" / password: "12345678"
3. Open browser and navigate to http://192.168.4.1

4. Select the STM32 .bin file and click Upload & Flash STM32
5. Wait 10-30 seconds for transfer to complete





6. Browser shows "Flash success!" — STM32 reboots into new firmware

Memory Optimization

The firmware binary is never loaded entirely into RAM. SPIFFS streaming keeps memory usage minimal:

Buffer	Size	Location
HTTP chunk buffer	512 bytes	Stack (temporary)
XModem block buffer	128 bytes	Stack (per block)
XModem packet	131 bytes	Stack (per block)

Buffer	Size	Location
SPIFFS file handle	~100 bytes	Heap
Firmware binary (old)	256 KB static	Removed — was cause of DRAM overflow

STM32 Custom Bootloader Requirements

The STM32 must be running a custom bootloader (not ST system bootloader) with the following characteristics:

Parameter	Value
UART peripheral	USART1 (huart1)
Baud rate	115200
XModem mode	Checksum (classic) — sends NAK 0x15
NAK_INIT_DELAY	3000 ms after reset
XMODEM_TIMEOUT_MS	3000 ms per byte
XMODEM_MAX_RETRIES	3
APP_START_ADDR	0x08004000 (Sector 1)
BOOT0 trigger	PA0 = HIGH at reset

Troubleshooting

Symptom	Cause	Fix
ESP32 resets in boot loop	GPIO 25/26 conflict with DAC	Use GPIO 32/33 instead
DRAM overflow at link	256 KB static fw_buf in BSS	Stream from SPIFFS, no static buffer
Garbage handshake bytes	BOOTLOADER_READY_MS too short	Set to 3500 ms (>= NAK_INIT_DELAY)
Block 1 always NAK/CAN	TX/RX lines swapped	Verify ESP TX -> STM32 RX wiring
Timeout on all handshake bytes	Wrong baud rate or UART port	Check 115200 baud, UART_NUM_2
IllegalInstruction panic	uart_driver_install called twice	Guard with uart_is_driver_installed()

API Reference

stm32_updater.h

Function	Description
void stm32_hw_init(void)	Initialize GPIO (BOOT0, NRST) and UART2. Call once in app_main().
esp_err_t stm32_enter_bootloader(void)	Assert BOOT0, pulse NRST, wait for bootloader ready.
esp_err_t stm32_exit_bootloader(void)	De-assert BOOT0, pulse NRST to reboot into application.
esp_err_t stm32_ota_from_spiffs(const char *path)	Full OTA sequence: enter bootloader, XModem stream from file, exit bootloader.

Dependencies

- ESP-IDF 5.x (via PlatformIO espressif32 platform)
- esp_wifi — Wi-Fi Access Point
- esp_http_server — HTTP upload server
- esp_spiffs / esp_vfs_spiffs — Filesystem for firmware storage
- driver/uart — UART2 for STM32 communication
- driver/gpio — BOOT0 and NRST control